Python Activity-Based Learning
Case Study

13 ATM Transaction Sequence Simulation

Selected case study | Prepared using the topics listed in the supplied assignment.

1. Problem Statement

Create a program that processes a sequence of deposits and withdrawals, maintains the account balance, rejects invalid transactions, enforces a withdrawal limit, and prints a transaction summary.

2. Objectives

• Use variables and numeric data types to maintain an account balance.

• Use loops and conditional statements to process multiple transactions.

• Validate transaction amounts and enforce a withdrawal limit.

• Summarize successful deposits, withdrawals, and rejected transactions.

3. Assumptions

• The opening balance is non-negative.

• Each transaction is entered as D for deposit or W for withdrawal, followed by an amount.

• A transaction amount must be greater than zero.

• A single withdrawal cannot exceed ■10,000 or the available balance.

4. Algorithm

1. Start with an opening balance and zeroed counters.

2. Read the number of transactions.

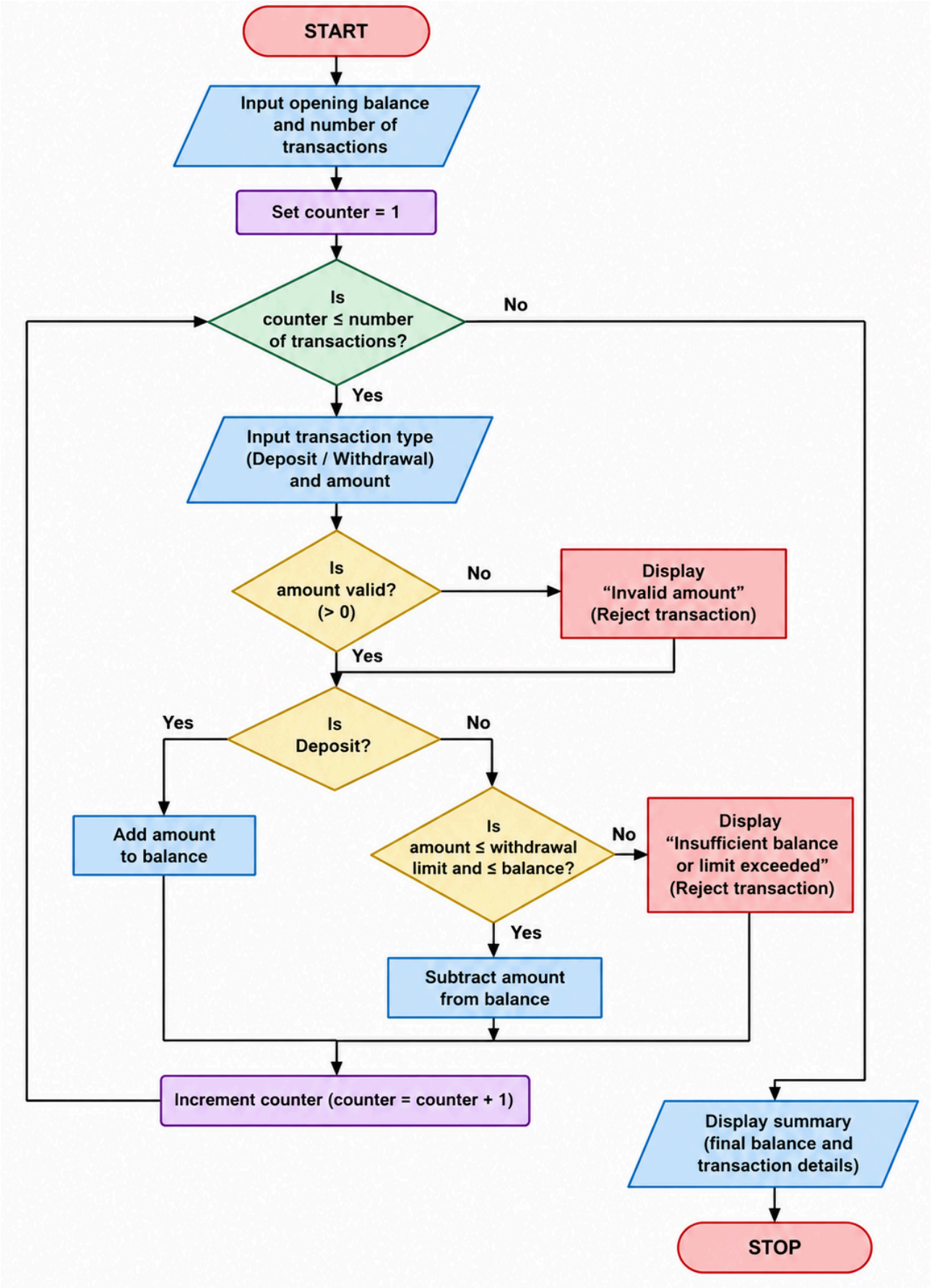
3. Repeat for each transaction: read its type and amount.

4. If the amount is invalid, count it as rejected.

5. For a deposit, add the amount to the balance and update the deposit total.

6. For a withdrawal, check the per-transaction limit and available balance; process it only if both conditions pass. 7. Display the final balance and transaction totals.

5. Flowchart



6. Pseudocode

```
BEGIN
```

```
 balance ← opening_balance
 deposits ← 0; withdrawals ← 0; rejected ← 0
 FOR each transaction
 INPUT type, amount
 IF amount <= 0 THEN rejected ← rejected + 1
 ELSE IF type = 'D' THEN
 balance ← balance + amount
 deposits ← deposits + amount
 ELSE IF type = 'W' AND amount <= 10000 AND amount <= balance THEN
 balance ← balance - amount
 withdrawals ← withdrawals + amount
 ELSE rejected ← rejected + 1
 END IF
 END FOR
 PRINT balance, deposits, withdrawals, rejected
END
```

Case Study 13 — ATM Transaction Sequence Simulation

7. Python Source Code

```
balance = float(input("Opening balance: "))
n = int(input("Number of transactions: "))

deposit_total = 0
withdrawal_total = 0
rejected = 0

for i in range(n):
 kind = input("Transaction type (D/W): ").upper()
 amount = float(input("Amount: "))

 if amount <= 0:
 print("Rejected: amount must be positive")
 rejected += 1
 elif kind == "D":
 balance += amount
 deposit_total += amount
 print("Deposit successful")
 elif kind == "W":
 if amount > 10000:
 print("Rejected: withdrawal limit is 10000")
 rejected += 1
 elif amount > balance:
 print("Rejected: insufficient balance")
 rejected += 1
 else:
 balance -= amount
 withdrawal_total += amount
 print("Withdrawal successful")
 else:
 print("Rejected: invalid transaction type")
 rejected += 1

print("\n--- TRANSACTION SUMMARY ---")
print("Final balance:", balance)
print("Total deposits:", deposit_total)
print("Total withdrawals:", withdrawal_total)
print("Rejected transactions:", rejected)
```

8. Test Cases

Input / Scenario	Expected result
Opening 5000; D 1000; W 2000	Final balance 4000; deposits 1000; withdrawals 2000; rejected 0
Opening 3000; W 4000	Final balance 3000; rejected 1
Opening 20000; W 12000	Final balance 20000; rejected 1

9. Conclusion

The program demonstrates transaction processing using loops, conditions, arithmetic operators, and validation. Note: Run the program in Python 3. The sample test cases are illustrative; enter the values when prompted.